

Nombre y Apellido:..... Curso:

TEORÍA					PRÁCTICA			NOTA
1	2	3	4	5	1	2	3	

TEORÍA: Responda brevemente las siguientes preguntas. Justifique.

1. Explique cómo la técnica de Prevención impide que se genere el deadlock y compárelo con Evasión en términos de overhead y flexibilidad a la hora de solicitar recursos.
2. Compare la utilización de procesos, hilos KLTs e hilos ULTs en términos de posibilidad de multiprocesamiento, overhead en su creación y seguridad.
3. Detalle qué sucede cuando ocurre una interrupción por fin de Quantum asumiendo que las interrupciones están habilitadas. ¿Podría ocurrir un cambio de Proceso a raíz de esto?
4. Dado un sistema con alto uso de CPU y condiciones de carrera muy frecuentes, explique si es más performante utilizar semáforos con bloqueo o una solución con test and set.
5. Responda Verdadero o Falso, justificando en ambos casos:

a. El planificador de largo plazo es el único que influye en el grado de multiprogramación.

b. HRRN es un algoritmo sin desalojo que evita la inanición.

PRÁCTICA: Resuelva los siguientes ejercicios justificando las conclusiones obtenidas.

Ejercicio 1

Existen 2 procesos que se encuentran en ready. Además, los recursos que se solicitan usar y las instancias que se disponen de estos son: 1 teclado, 1 mouse y 2 monitores.

Ambos procesos comparten una biblioteca de manejo de recursos que provee las funciones *recursos_disponibles*, *usar_recurso* y *liberar_recursos*.

recursos_disponibles consulta una variable global **RECURSOS_ACTUALES** y retorna true si están todos disponibles. Tanto *usar_recurso* como *liberar_recursos* modifican **RECURSOS_ACTUALES**.

usar_recurso es bloqueante si el recurso no se encuentra disponible.

Proceso 1	Proceso 2
<pre>while (true) { char* rec_a_usar = [TECLADO, MOUSE] if(recursos_disponibles(rec_a_usar) { usar_recurso(rec_a_usar[0]); usar_recurso(rec_a_usar[1]); liberar_recursos(rec_a_usar); } }</pre>	<pre>while (true) { char* rec_a_usar = [MOUSE, MONITOR, TECLADO] if (recursos_disponibles(rec_a_usar) { usar_recurso(rec_a_usar[0]); usar_recurso(rec_a_usar[1]); usar_recurso(rec_a_usar[2]); liberar_recursos(rec_a_usar); } }</pre>

Responda: ¿Es posible garantizar que los procesos puedan ejecutar sin inconvenientes? De no ser así, mencione que posibles problemas pueden ocurrir y al menos 2 soluciones a los mismos. De finalizar correctamente, mencione si la ejecución podría ser no determinística.

Ejercicio 2

En un laboratorio donde trabaja 1 revolucionaria bioquímica, se intenta recrear la estructura de ciertas proteínas. Se tiene N recolectores que se encargan de hacer el trabajo de campo buscando muestras de diversos seres vivos. Los mismos cuentan con un repositorio con capacidad para 20 muestras. La bioquímica luego las procesa y envía su reporte a los analistas.

Existen 3 analistas, cada uno con una especialidad particular, que deben revisar el reporte de la bioquímica y sacar sus conclusiones en base a su especialidad. Es fundamental que las muestras sean analizadas por los 3 analistas. Nota: getIdEspecialidad() devuelve el id del analista (0, 1 o 2).

Analista (3 instancias)	Recolector de Muestras (N instancias)
<pre>while(1) { id_especialidad = getIdEspecialidad(); revisar_reporte(); }</pre>	<pre>while(1) { muestra = recolectar_muestra(); depositar(muestra, repositorio_muestras); }</pre>
Bioquímica (1 instancia)	
<pre>while(1) { muestra = retirar_muestra(repositorio_muestras); procesar_muestra(muestra); publicar_reporte(); }</pre>	

Ejercicio 3

Se tiene un SO con un planificador a corto plazo que implementa el algoritmo Round Robin con un quantum=3. Además, algunos utilizan una biblioteca de ULTs, planificada mediante SJF (sin desalojo).

Por otra parte, al momento que se crean nuevos Procesos, el SO tiene una política en la que, si el nivel máximo de multiprogramación (seteado en 2 Procesos) es alcanzado y el nuevo Proceso tiene mayor prioridad que algunos de los existentes en Ready, el nuevo proceso ingresará, suspendiendo (retirando de memoria principal) al de menor prioridad.

Nota: La prioridad se mide en forma descendente, es decir, 0 es la máxima prioridad.

PROCESOS			PRIORIDAD	ARRIBO	CPU	IO	CPU
PA	KLT1		3	0	2	5	1
	KLT2	ULT1		0	2	2	2
		ULT2		3	1	1	1
PB	KLT3		2	2	4	1	1
PC	KLT4	ULT3	0	6	2	3	4
		ULT4		6	3	1	1
PD	KLT5		1	9	5	-	-

- a) Realice el diagrama de GANTT según la traza de ejecución que muestra la tabla.
- b) Indique los instantes en los cuales ocurrieron syscalls. ¿Cuáles fueron los motivos?

Condiciones de aprobación: 3 preguntas correctamente respondidas y 1.5 ejercicios correctamente resueltos.

